

Low-Level Design (LLD)

Multi-Acquirer Payment Gateway — Spring Boot Microservice

Version: 1.0

Date: 2026-06-08

1. Project Structure

```

payment-gateway/
├── src/
│   ├── main/
│   │   ├── java/com/worldwide/paymentgateway/
│   │   │   ├── PaymentGatewayApplication.java
│   │   │   └── config/
│   │   │       ├── AcquirerProperties.java      # Acquirer config binding
│   │   │       ├── DatabaseConfig.java         # JPA / DataSource
│   │   │       ├── RedisConfig.java            # Lettuce Redis client
│   │   │       ├── ResilienceConfig.java       # Resilience4j beans
│   │   │       ├── SecurityConfig.java         # Spring Security
│   │   │       └── WebClientConfig.java        # HTTP client pools
│   │   └── api/
│   │       ├── controller/
│   │       │   ├── PaymentController.java     # POST /v1/payments
│   │       │   └── HealthController.java      # GET /health
│   │       ├── dto/
│   │       │   ├── request/
│   │       │   │   ├── PaymentRequest.java
│   │       │   │   ├── CaptureRequest.java
│   │       │   │   └── RefundRequest.java
│   │       │   └── response/
│   │       │       ├── PaymentResponse.java
│   │       │       └── ErrorResponse.java
│   │       ├── validation/
│   │       │   └── CurrencyValidator.java
│   │       └── domain/
│   │           ├── entity/
│   │           │   ├── Transaction.java       # JPA entity
│   │           │   ├── AcquirerConfig.java    # JPA entity
│   │           │   └── RoutingRule.java        # JPA entity
│   │           ├── enums/
│   │           │   ├── TransactionStatus.java
│   │           │   ├── AcquirerType.java
│   │           │   └── CardNetwork.java
│   │           ├── repository/
│   │           │   ├── TransactionRepository.java
│   │           │   ├── AcquirerConfigRepository.java
│   │           │   └── RoutingRuleRepository.java
│   │           └── service/
│   │               ├── PaymentService.java     # Core orchestrator
│   │               ├── IdempotencyService.java # Redis-backed
│   │               ├── FraudDetectionService.java
│   │               ├── routing/
│   │               │   └── AcquirerRoutingService.java
│   │               ├── notification/
│   │               │   └── KafkaNotificationService.java
│   │               └── acquirer/
│   │                   ├── AcquirerConnector.java # Interface
│   │                   ├── AcquirerRequest.java
│   │                   ├── AcquirerResponse.java
│   │                   └── impl/
│   │                       ├── StripeConnector.java
│   │                       ├── AdyenConnector.java
│   │                       └── BraintreeConnector.java

```

```
|
|
|
|
|   └─ WorldpayConnector.java
|   └─ factory/
|       └─ AcquirerConnectorFactory.java
|   └─ security/
|       └─ TokenizationService.java
|   └─ exception/
|       ├── PaymentException.java
|       ├── AcquirerException.java
|       ├── IdempotencyConflictException.java
|       └─ GlobalExceptionHandler.java
|   └─ resources/
|       ├── application.yml
|       ├── application-prod.yml
|       └─ db/migration/
|           ├── V1_create_transactions.sql
|           ├── V2_create_acquirer_configs.sql
|           └─ V3_create_routing_rules.sql
|   └─ test/
|       ├── java/com/worldwide/paymentgateway/
|       │   ├── service/
|       │   │   ├── PaymentServiceTest.java
|       │   │   └─ AcquirerRoutingServiceTest.java
|       │   └─ controller/
|       │       └─ PaymentControllerTest.java
|   └─ k8s/
|       ├── deployment.yaml
|       ├── service.yaml
|       ├── ingress.yaml
|       ├── configmap.yaml
|       ├── hpa.yaml
|       └─ pdb.yaml
|   └─ Dockerfile
|   └─ pom.xml
```

2. API Design

2.1 Endpoints

Method	Path	Description	Auth
POST	/v1/payments	Create authorization	API Key
POST	/v1/payments/{id}/capture	Capture authorized payment	API Key
POST	/v1/payments/{id}/refund	Refund a payment	API Key
POST	/v1/payments/{id}/void	Void authorization	API Key
GET	/v1/payments/{id}	Get payment status	API Key
GET	/actuator/health	Health check	None
GET	/actuator/metrics	Prometheus metrics	Internal

2.2 Request / Response Schemas

POST /v1/payments (Authorization Request)

```
{
  "idempotency_key": "ord_abc123_attempt1",
  "merchant_id": "merch_xxxx",
  "amount": 10050,
  "currency": "USD",
  "payment_method": {
    "type": "card",
    "token": "tok_visa_4242",
    "card_network": "VISA",
    "expiry_month": 12,
    "expiry_year": 2028
  },
  "capture_mode": "AUTOMATIC",
  "metadata": {
    "order_id": "ORD-9876",
    "customer_id": "cust_555"
  }
}
```

Response (201 Created)

```
{
  "id": "txn_a1b2c3d4",
  "status": "AUTHORIZED",
  "amount": 10050,
  "currency": "USD",
  "acquirer": "STRIPE",
  "acquirer_transaction_id": "pi_stripe_xyz",
  "created_at": "2026-06-08T10:00:00Z"
}
```

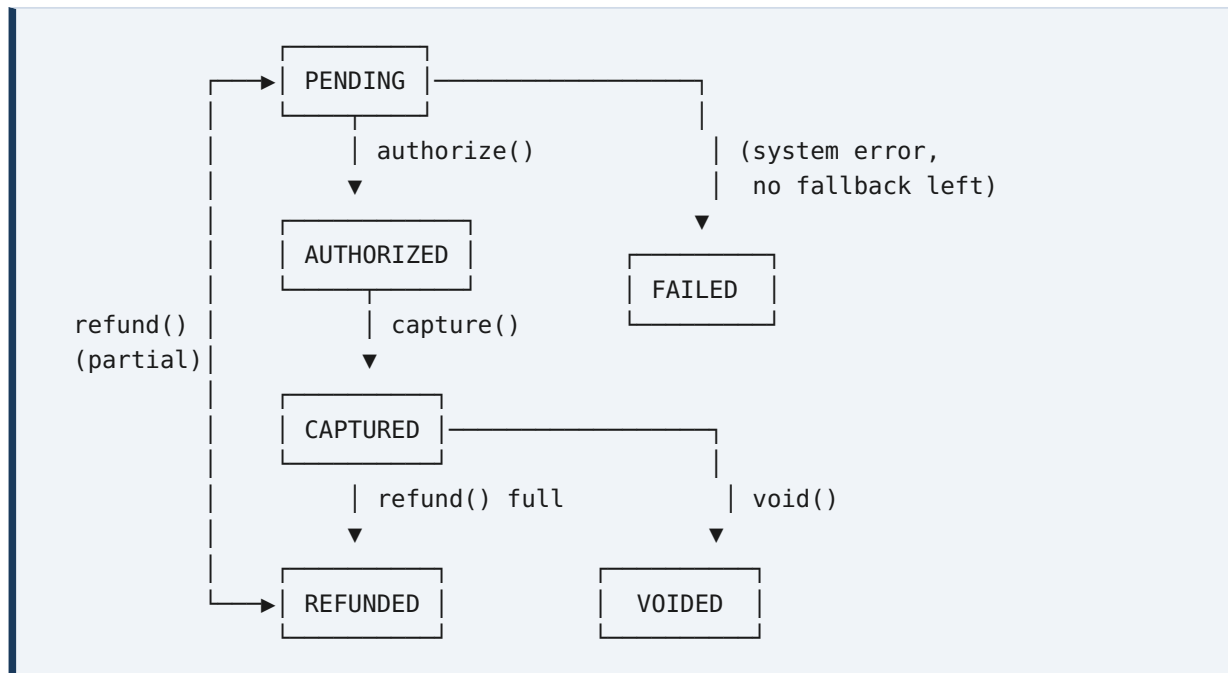
3. Domain Model

3.1 Transaction Entity

Transaction

id	: UUID (PK, generated)
merchantId	: String (indexed)
idempotencyKey	: String (unique index)
status	: TransactionStatus (enum)
amount	: Long (minor currency units, e.g. cents)
currency	: String (ISO 4217, 3-char)
cardToken	: String (tokenized PAN reference)
cardNetwork	: CardNetwork (enum)
captureMode	: CaptureMode (enum)
acquirerType	: AcquirerType (enum)
acquirerTxnId	: String
failureCode	: String (nullable)
failureMessage	: String (nullable)
fraudScore	: Integer (0-100)
metadata	: JSONB
createdAt	: OffsetDateTime
updatedAt	: OffsetDateTime
version	: Long (optimistic lock)

3.2 TransactionStatus State Machine



4. Service Design

4.1 PaymentService — Authorization Flow

`PaymentService.authorize(PaymentRequest req):`

1. Validate request (bean validation + custom rules)
2. `idempotencyService.check(req.idempotencyKey)`
 - if found: return cached response (HTTP 200)
3. `fraudDetectionService.score(req) → score`
 - if `score > threshold`: reject (FRAUD_BLOCKED)
4. `acquirerRoutingService.getOrderedAcquirers(req) → [A1, A2, A3]`
5. For each acquirer in list:
 - a. `connector = factory.get(acquirer)`
 - b. `acquirerReq = map(req) to AcquirerRequest`
 - c. Try: `acquirerResp = connector.authorize(acquirerReq)`
 - [Circuit Breaker + Timeout wrapping]
 - On `AcquirerSystemException`: continue to next acquirer
 - On `AcquirerDeclineException`: STOP (hard decline)
 - On Success: break loop
6. Save Transaction to DB (status = AUTHORIZED | FAILED)
7. `idempotencyService.store(req.idempotencyKey, txn.id)`
8. `kafkaNotificationService.publish(txn)`
9. Return `PaymentResponse`

4.2 AcquirerRoutingService

```
getOrderedAcquirers(PaymentRequest req) → List<AcquirerType>:

1. Load all enabled AcquirerConfigs from cache/DB
2. Filter by RoutingRules matching:
   - req.cardNetwork
   - req.currency
   - req.amount (min/max range)
3. For each candidate acquirer, compute score:
   score = (successRate × 0.5) + (1/normalizedCost × 0.3) + (1/latency × 0.2)
4. Sort descending by score
5. Return ordered list (first = primary, rest = fallbacks)
```

4.3 IdempotencyService

```
check(key: String) → Optional<Transaction>:
  Redis GET "idempotency:{md5(key)}"
  → deserialize Transaction if found

store(key: String, txnId: UUID):
  Redis SETEX "idempotency:{md5(key)}" 86400 txnId
  (TTL = 24 hours)
```

4.4 FraudDetectionService

```
score(req: PaymentRequest) → int (0–100):

Rules (additive):
- amount > $5,000 → +20
- cardToken flagged in blocklist (Redis SET) → +60
- >3 attempts from same merchantId in 60s → +30
- currency mismatch with card BIN region → +15

Final score capped at 100.
Threshold: reject if score ≥ 70 (configurable).
```


5. Acquirer Connector Design

5.1 Interface

```
public interface AcquirerConnector {  
    AcquirerType getType();  
    AcquirerResponse authorize(AcquirerRequest request);  
    AcquirerResponse capture(String acquirerTxnId, long amount, String  
currency);  
    AcquirerResponse refund(String acquirerTxnId, long amount, String currency);  
    AcquirerResponse voidTransaction(String acquirerTxnId);  
}
```

5.2 Error Classification

Exception Type	Meaning	Action
<code>AcquirerDeclineException</code>	Hard decline (NSF, stolen, blocked)	Stop routing; return FAILED
<code>AcquirerSystemException</code>	Acquirer system/network error	Try next acquirer (failover)
<code>AcquirerTimeoutException</code>	Response timeout exceeded	Try next acquirer (failover)

5.3 Circuit Breaker (per acquirer)

State transitions:

- CLOSED → OPEN: 5 failures in 10-second sliding window
- OPEN → HALF_OPEN: after 30-second wait
- HALF_OPEN → CLOSED: 3 consecutive successes
- HALF_OPEN → OPEN: 1 failure

6. Database Schema

V1 — Transactions

```
CREATE TABLE transactions (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  merchant_id       VARCHAR(64) NOT NULL,  
  idempotency_key   VARCHAR(255) NOT NULL,  
  status            VARCHAR(32) NOT NULL,  
  amount            BIGINT      NOT NULL,  
  currency          CHAR(3)     NOT NULL,  
  card_token        VARCHAR(255),  
  card_network      VARCHAR(32),  
  capture_mode      VARCHAR(32) NOT NULL DEFAULT 'AUTOMATIC',  
  acquirer_type     VARCHAR(64),  
  acquirer_txn_id   VARCHAR(255),  
  failure_code      VARCHAR(64),  
  failure_message   TEXT,  
  fraud_score       SMALLINT     NOT NULL DEFAULT 0,  
  metadata          JSONB,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  version           BIGINT      NOT NULL DEFAULT 0,  
  CONSTRAINT uq_idempotency_key UNIQUE (idempotency_key)  
);  
  
CREATE INDEX idx_transactions_merchant_id ON transactions (merchant_id);  
CREATE INDEX idx_transactions_status     ON transactions (status);  
CREATE INDEX idx_transactions_created_at  ON transactions (created_at DESC);
```

V2 — Acquirer Configs

```
CREATE TABLE acquirer_configs (  
  id                SERIAL PRIMARY KEY,  
  acquirer_type     VARCHAR(64) NOT NULL UNIQUE,  
  endpoint_url      VARCHAR(512) NOT NULL,  
  api_key_ref       VARCHAR(255) NOT NULL, -- AWS Secrets Manager ARN  
  enabled           BOOLEAN     NOT NULL DEFAULT TRUE,  
  priority          INT         NOT NULL DEFAULT 100,  
  success_rate      DECIMAL(5,4) NOT NULL DEFAULT 1.0,  
  avg_latency_ms    INT         NOT NULL DEFAULT 500,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

V3 — Routing Rules

```
CREATE TABLE routing_rules (  
  id          SERIAL PRIMARY KEY,  
  acquirer_id INT          NOT NULL REFERENCES acquirer_configs(id),  
  priority    INT          NOT NULL DEFAULT 100,  
  card_network VARCHAR(32), -- NULL = any  
  currency    CHAR(3),      -- NULL = any  
  min_amount  BIGINT,        -- NULL = no lower bound  
  max_amount  BIGINT,        -- NULL = no upper bound  
  enabled     BOOLEAN       NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_routing_rules_acquirer_id ON routing_rules (acquirer_id);
```

7. Configuration

application.yml (non-sensitive)

```

server:
  port: 8080

spring:
  application:
    name: payment-gateway
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:5432/${DB_NAME:paymentdb}
    username: ${DB_USER:pguser}
    password: ${DB_PASSWORD}          # injected from Secrets Manager
  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
    locations: classpath:db/migration
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: 6379
      password: ${REDIS_PASSWORD:}
      lettuce:
        pool:
          max-active: 20
    kafka:
      bootstrap-servers: ${KAFKA_BROKERS:localhost:9092}
      producer:
        key-serializer: org.apache.kafka.common.serialization.StringSerializer
        value-serializer:
org.springframework.kafka.support.serializer.JsonSerializer

payment:
  fraud:
    score-threshold: 70
    velocity-window-seconds: 60
    velocity-max-attempts: 3
  idempotency:
    ttl-seconds: 86400

acquirers:
  stripe:
    endpoint-url: https://api.stripe.com
    api-key-ref: arn:aws:secretsmanager:us-east-1:ACCT:secret:stripe-api-key
    enabled: true
    priority: 1
  adyen:
    endpoint-url: https://checkout-test.adyen.com
    api-key-ref: arn:aws:secretsmanager:us-east-1:ACCT:secret:adyen-api-key
    enabled: true
    priority: 2
  worldpay:
    endpoint-url: https://api.worldpay.com

```

```
api-key-ref: arn:aws:secretsmanager:us-east-1:ACCT:secret:worldpay-api-key
enabled: true
priority: 3

resilience4j:
  circuitbreaker:
    instances:
      stripe:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 3
      adyen:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
  retry:
    instances:
      acquirer-retry:
        max-attempts: 2
        wait-duration: 100ms
        exponential-backoff-multiplier: 2
  timelimiter:
    instances:
      acquirer-timeout:
        timeout-duration: 3s
```

8. Kubernetes Manifests

deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-gateway
  namespace: payment-gateway-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment-gateway
  template:
    metadata:
      labels:
        app: payment-gateway
    spec:
      serviceAccountName: payment-gateway-sa # IRSA role
      containers:
        - name: payment-gateway
          image: <ECR_REPO>/payment-gateway:latest
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "500m"
              memory: "512Mi"
            limits:
              cpu: "2000m"
              memory: "1Gi"
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            initialDelaySeconds: 20
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 15
          env:
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: payment-gateway-config
                  key: db_host
            - name: REDIS_HOST
              valueFrom:
                configMapKeyRef:
                  name: payment-gateway-config
                  key: redis_host
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:

```



```
name: payment-gateway-secrets
key: db_password
```

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payment-gateway-hpa
  namespace: payment-gateway-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-gateway
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

9. Testing Strategy

Test Type	Framework	Coverage Target	Scope
Unit tests	JUnit 5 + Mockito	80% line coverage	Service logic, routing algorithm
Integration tests	Spring Boot Test + Testcontainers	Critical paths	DB, Redis, Kafka interactions
Contract tests	Spring Cloud Contract	All acquirer connectors	Request/response mapping
Load tests	Gatling	1,000 TPS	End-to-end with mock acquirers